1 THE STEERING TURRET CORAL SOFTWARE

1.1 CONVENTIONS:

```
> linux commands
```

```
C:\windows path or files
/linux path or files
```

```
Terminal output
Example of focus items
>
```

1.2 BASE KNOWLEDGE

A **controller area network bus** (**CAN bus**) is a vehicle bus standard designed to enable efficient communication primarily between electronic control units (ECUs). Originally developed to reduce the complexity and cost of electrical wiring in automobiles through multiplexing, the CAN bus protocol has since been adopted in various other contexts. This broadcast-based, message-oriented protocol ensures data integrity and prioritization through a process called arbitration, allowing the highest priority device to continue transmitting if multiple devices attempt to send data simultaneously, while others back off. Its reliability is enhanced by differential signaling, which mitigates electrical noise.

CAN is regulated by the following International Organization for Standardization (ISO) standards:

- ISO 11898-1 which covers the data link layer,
- ISO 11898-2 which covers the CAN physical layer for high-speed CAN.

The CAN bus is used to control all moving parts of the ST (motors, rotary stages, etc) and any other device that can be controlled through a CAN bus.

JSON (JavaScript Object Notation) is a lightweight data-interchange format. It is easy for humans to read and write. It is easy for machines to parse and generate. It is based on a subset of the JavaScript Programming Language Standard ECMA-262 3rd Edition - December 1999. JSON is a text format that is completely language independent but uses conventions that are familiar to programmers of the C-family of languages, including C, C++, C#, Java, JavaScript, Perl, Python, and many others. These properties make JSON an ideal data-interchange language.

JSON is built on two structures:

- A collection of name/value pairs. In various languages, this is realized as an *object*, record, struct, dictionary, hash table, keyed list, or associative array.
- An ordered list of values. In most languages, this is realized as an *array*, vector, list, or sequence.

These are universal data structures. Virtually all modern programming languages support them in one form or another. It makes sense that a data format that is interchangeable with programming languages also be based on these structures.

The JSON protocol over UDP is used between the Control Unit applications and the STC control application for Commands and Telemetries.

RS-232 or Recommended Standard 232 is a standard originally introduced for serial communication transmission of data. It formally defines signals connecting between a DTE (data terminal equipment) such as a computer terminal or PC, and a DCE (data circuit-terminating equipment or data communication equipment), such as a modem. The standard defines the electrical characteristics and timing of signals, the meaning of signals, and the physical size and pinout of connectors. The current version of the standard is TIA-232-F Interface Between Data Terminal Equipment and Data Circuit-Terminating Equipment Employing Serial Binary Data Interchange.

RS-232 is used in communications between the STC Control board and the dual antenna GPS receiver

NMEA 0183 is a combined electrical and data specification for communication between marine electronics such as echo sounder, sonars, anemometer, gyrocompass, GPS receivers and many other types of instruments. It has been defined and is controlled by the National Marine Electronics Association (NMEA). The electrical standard that is used is EIA-422, also known as RS-422, although most hardware with NMEA-0183 outputs are also able to drive a single EIA-232 port. The standard calls for optically isolated inputs. There is no requirement for isolation for the outputs.

RS-232 is used in communications between the STC Control board and the dual antenna GPS receiver. It also includes some specific manufacturer sentences.

The **Pine64 RockPro64** is the most powerful single-board computer released by PINE64. It is powered by a Rockchip RK3399 hexa-core (dual ARM Cortex A72 and quad ARM Cortex A53) 64-bit processor with a Mali T-860 quad-core GPU. The key features include a PCI Express (PCIe) x4 open-ended slot, the use of LPDDR4 DRAM, and industry-standard heatsink mounting holes.

The ROCKPro64 is equipped with 2 or 4 GB of LPDDR4 system memory, and 128 Mb of SPI boot flash. There is also an optional eMMC module (up to 128 GB) and microSD slot for booting. The board is equipped with 1x USB 3.0 Type-C host port with DP 1.2, 1x USB 3.0 Type-A host port, 2x USB 2.0 host port, Gigabit Ethernet, PI-2 GPIO bus, MiPi DSI interface, eDP interface, touch panel interface, stereo MiPi CSI interface, as well as many other device interfaces such as UART, SPI, I2C, for makers to integrate with sensors and other peripherals. Many different operating systems (OSes) are freely available from the open-source community, such as Linux (Ubuntu, Debian, Arch), BSD and Android.

Manjaro Arch Linux is a user-friendly Linux distribution based on the independently developed Arch operating system. Within the Linux community, Arch itself is renowned for being an exceptionally fast, powerful, and lightweight distribution that provides access to the very latest cutting-edge and bleeding-edge software. However, Arch is also aimed at more experienced or technically-minded users. As such, it is generally considered to be beyond the reach of those who lack the technical expertise (or persistence) required to use it.

Developed in Austria, France, and Germany, Manjaro provides all the benefits of the Arch operating system combined with a focus on *user-friendliness* and *accessibility*. Manjaro follows Arch Linux and officially only offers a 64-bit version.

2 CONTROL UNIT DEVELOPMENT PLATFORM

CPU:	description
Operating System:	description
Language:	English
IDE:	description

2.1 TOOLCHAIN

The `build-essential` package is a meta-package that includes a collection of essential software tools and libraries required for building and compiling software from source code. It's a convenient way to install a set of common build tools, ensuring that you have the necessary components to compile and build software on your Linux system.

The package typically includes:

1. *GNU Compiler Collection (gcc)*: the essential compiler for C, C++, and other programming languages.
2. *GNU Make*: a build automation tool that manages the build process of a project.
3. *libc6-dev*: development libraries and header files for the C library.
4. *dpkg-dev*: tools for building Debian source packages.
5. *GNU Binutils*: a collection of binary tools, including the linker and assembler.

```
> sudo sudo apt install build-essential
```

2.2 TOOLS AND LIBRARIES

2.3 DRIVERS

2.4 COMPILE AND LINK THE CU PROJECT

2.5 CREATE THE INSTALLATION/UPDATE DATA-SET

3 CONTROL UNIT OPERATIONAL PLATFORM

3.1 HW/SW PRE-REQUISITES

3.2 OS INSTALLATIONS AND CONFIGURATION

3.3 SET UP THE CONTROL UNIT PLATFORM

3.4 ADDITIONAL INSTALLATIONS

3.5 INSTALL THE CU SW

3.6 UPDATE THE CU SW

4 STC DEVELOPMENT PLATFORM

The STC Control application was developed on a platform having the following setup. Any Linux distribution and the preferred IDE can be used.

It is only suggested to use the recommended compiler for ARM A53/A72.

Minimum HW requirements

MCU:	any Intel/AMD
RAM:	8 GB
HD free space	100 MB

Minimum SW requirements

Operating System:	Ubuntu 22.04.x LTS
Language:	English
Username:	*whatever* alma
Password:	according internal policy alma2024
IDE:	Eclipse
	Download eclipse from www.eclipse.org and run the installer. Select the "Eclipse IDE for Embedded C/C++ Developers".

GCC compiler

- Intel gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0
- ARM Cortex-A53 aarch64-linux-gnu-gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0

4.1 OS INSTALLATIONS AND CONFIGURATION

4.2 TOOLCHAIN FOR INTEL/AMD

The `build-essential` package is a meta-package that includes a collection of essential software tools and libraries required for building and compiling software from source code. It's a convenient way to install a set of common build tools, ensuring that you have the necessary components to compile and build software on your Linux system.

The package typically includes:

1. *GNU Compiler Collection (gcc)*: the essential compiler for C, C++, and other programming languages.
2. *GNU Make*: a build automation tool that manages the build process of a project.
3. *libc6-dev*: development libraries and header files for the C library.
4. *dpkg-dev*: tools for building Debian source packages.
5. *GNU Binutils*: a collection of binary tools, including the linker and assembler.

The gcc (Ubuntu 11.4.0-1ubuntu1~22.04) 11.4.0 was used for the development platform.

```
> sudo sudo apt install build-essential
```

4.2.1 Compiler for ARM 32/64 bit

Dependency package providing the default GNU C cross-compiler (64-bit) for the arm64 architecture.

4.2.1.1 gcc-aarch64-linux-gnu

```
> sudo apt-get install gcc-aarch64-linux-gnu
```

4.2.1.2 gcc-arm-linux-gnueabihf (optional)

```
> sudo apt-get install gcc-arm-linux-gnueabihf
```

`gcc-arm-linux-gnueabihf` builds binaries for ARM architecture running linux OS and running GNU Embedded Application Binary Interface (EABI). Use this release to cross compile user space applications and run it on an ARM target running Embedded Linux OS.

For most embedded ARMs the arm-none-eabi toolset is fine.

4.3 TOOLS AND LIBRARIES

4.3.1 USB management

`usbutils` is a collection various utilities for querying the the Universal Serial Bus (USB).

```
> sudo snap install usbutils-snap
```

4.3.2 CAN management

SocketCAN userspace utilities and tools contains some userspace utilities for Linux CAN subsystem (aka SocketCAN). It includes:

1. Basic tools to display, record, generate and replay CAN traffic
   - candump: display, filter and log CAN data to files
   - canplayer: replay CAN logfiles
   - cansend: send a single frame
   - cangen: generate (random) CAN traffic
   - cansequence: send and check sequence of CAN frames with incrementing payload
   - cansniffer: display CAN data content differences
2. CAN access via IP sockets
   - canlogserver: log CAN frames and serves them
   - bcmserver: interactive BCM configuration (remote/local)
   - socketcand: use RAW/BCM/ISO-TP sockets via TCP/IP sockets
   - cannelloni: UDP/SCTP based SocketCAN tunnel
3. CAN in-kernel gateway configuration
   - cangw: CAN gateway userspace tool for netlink configuration
4. CAN bus measurement and testing
   - canbusload: calculate and display the CAN busload
   - can-calc-bit-timing: userspace version of in-kernel bitrate calculation
   - canfdtest: Full-duplex test program (DUT and host part)
   - canerrsim: CAN error message simulator

To install SocketCAN, run

```
> sudo apt-get -y install can-utils
```

4.3.3 Network Management

`net-utils` package includes the important tools for controlling the network subsystem of the Linux kernel. This includes arp, ifconfig, netstat, rarp, nameif and route. Additionally, this package contains utilities relating to particular network hardware types (plipconfig, slattach, mii-tool) and advanced aspects of IP configuration (iptunnel, ipmaddr). To install the network tools, run:

```
> sudo apt-get install net-utils
```

4.3.4 Java runtime environment (optional)

The Java Runtime Environment (JRE) and the JVM (Java's virtual machine) are required to run Java programs. Here it is necessary for using the Eclipse IDE. This installation can be omitted if a different IDE will be used.

```
> sudo apt install default-jre
```

4.3.5 Unzip

`unzip` is required to list, test and extract compressed files in a ZIP archive. To install unzip (if not already installed), run

```
> sudo apt-get install unzip
```

4.3.6 Libgpio v1.6.3

`libgpio` is required to list, test and extract compressed files in a ZIP archive. To install unzip (if not already installed), run

```
> sudo apt install gpiod libgpiod-dev libgpiod-doc
```

Scarica la versione 2.1.1
wget https://mirrors.edge.kernel.org/pub/software/libs/libgpiod/libgpiod-2.1.1.tar.gz
tar xf libgpiod-2.1.1.tar.gz
cd libgpiod-2.1.1

tar tf libgpiod-2.1.1.tar.gz | head -20

compilazione e installazione
./autogen.sh
./configure --host=aarch64-linux-gnu --prefix=$PWD/stage
make
make install
#This builds it for ARM and installs to ./stage.

export SYSROOT=$PWD/stage

Per sapere quale versione è installata
grep GPIOD_VERSION $SYSROOT/include/gpiod.h

/home/alma/STCApps/libgpiod-1.6.3/stage/lib

4.4 IDE – ECLIPSE

The Eclipse CDT™ Project provides a fully functional C and C++ Integrated Development Environment based on the Eclipse platform. Features include: support for project creation and managed build for various toolchains, standard make build, source navigation, various source knowledge tools, such as type hierarchy, call graph, include browser, macro definition browser, code editor with syntax highlighting, folding and hyperlink navigation, source code refactoring and code generation, visual debugging tools, including memory, registers, and disassembly viewers.

The use of Eclipse IDE is not mandatory; any other IDE can be used instead. Compiler and Linker options are described in § 4.6.

For installing and configuring Eclipse IDE, see Annex 2 to this document.

4.5 DRIVERS

- **Waveshare USB-CAN adapter**

  Insert to device and identify the QinHeng Electronics CH340 serial converter ID

```
lsusb
Bus 001 Device 006: ID 0bda:0129 Realtek Semiconductor Corp. RTS5129 Card Reader Controller
Bus 001 Device 007: ID 0cf3:0036 Qualcomm Atheros Communications AR9462 Bluetooth
Bus 001 Device 004: ID 0bda:5756 Realtek Semiconductor Corp. Integrated_Webcam_HD
Bus 001 Device 009: ID 1a86:7523 QinHeng Electronics CH340 serial converter
Bus 001 Device 003: ID 0000:3825   USB OPTICAL MOUSE
Bus 001 Device 002: ID 8087:8000 Intel Corp. Integrated Rate Matching Hub
Bus 001 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
Bus 003 Device 001: ID 1d6b:0003 Linux Foundation 3.0 root hub
Bus 002 Device 002: ID 17ef:608c Lenovo Lenovo Calliope USB Keyboard
Bus 002 Device 001: ID 1d6b:0002 Linux Foundation 2.0 root hub
```

```
sudo modprobe option
sudo sh -c 'echo "1a86 7523" > /sys/bus/usb-serial/drivers/option1/new_id'
```

- **Waveshare USB-CAN adapter**

  https://www.waveshare.com/wiki/USB-CAN-A

```
lsusb
ls /dev
sudo su
sudo modprobe option
sudo sh -c 'echo "1a86 7523" > /sys/bus/usb-serial/drivers/option1/new_id'
```

```
sudo usermod -a -G dialout $USER
```

```
wget https://files.waveshare.com/upload/7/72/USB-CAN-A.zip
unzip USB-CAN-A.zip
cd USB-CAN-A
sudo make clean
sudo make
```

4.6 COMPILE AND LINK THE STC PROJECT

TBW

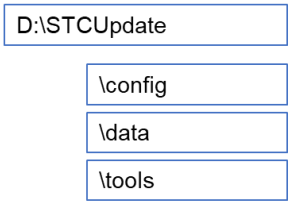
4.7 CREATE THE UPDATE TOOL-SET

Use a USB card to copy the updated applications and data files to the STC control board. On the USB card create the folder

```
C:\STCUpdate
```

and copy inside only the applications and files that are part of the update.

Follow instructions in § 5.5.



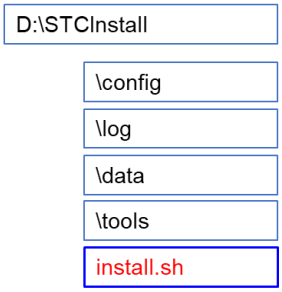
4.8 CREATE THE INSTALLATION TOOL-SET

Use a USB card to copy the new application to the STC control board. On the USB card create the folder

```
C:\STCInstall
```

and copy inside only the applications and files that are part of the update.

Follow instructions in § 5.5.



4.9 ADDITIONAL INSTALLATION

4.9.1 Connection to NAS (optional)

The connection to a NAS can be useful for data configuration and to share disk area during design, development, test and documentation activities.

To share a disk area, setup the sharing on the server (it could be a Windows PC, a NAS or another Linux server), then follow the steps:

1. Install CIFS filesystem

`cifs-utils` implements the SMB/CIFS protocol, that is a standard file sharing protocol widely deployed on Microsoft Windows machines. This package contains tools for mounting shares on Linux using the SMB/CIFS protocol.

```
> sudo apt-get install cifs-utils
```

2. Create a mount point on the local file system

```
> sudo mkdir /mnt/NAS
```

3. Set a permanent configuration, in order to mount the shared disk area at the start-up edit the

   `/etc/fstab`

   file and add following row for the partition to mount

```
//ServerIP/Shared /mnt/NAS cifs rw,auto,user=username,password=password  0  2
```

4. file and add following row for the partition to mount

```
//  ServerIP/sharedfolder /mnt/NAS cifs users,rw,auto,user=username,password=
password,gid=groupid,forcegid,file_mode=0775,dir_mode=0775  0  2
```

   where:

   serverIP is the IP of the server computer or NAS sharing the disk area;

   username/password are the account on the server computer or NAS, having sufficient grant to access, read and write data into the shared disk area.

   sharefolder is the NAS remote directory to be shared (it must exist).

   groupid is the group id the ST users belong to. This is required to allow a group of users to access in write mode. To get the correct group id type:

```
> id username
```

   and get the id of the users group `username` belongs to.

5. Create a Desktop link to the shared area

```
cd /mnt/NAS
ln -s $PWD ~/Desktop/
```

5 STC OPERATIONAL PLATFORM

The STC Control application runs on a Pine64 RockPro64 SBC. Following is the STC control board HW/SWsetup:

MCU:	Pine64 RockPro64 - Dual-core Cortex-A72 up to 2.0GHz CPU, implementing the full ARMv8-A 64-bit instruction set.
Operating System:	Manjaro 5.18 - Minimal ARM Installation (Kernel 5.18.5-1-MANJARO-ARM)
SD Memory Card:	32 GB microSD HC - Class 10, U1 (speed rate10 MB/s), providing OS and ST applications (see 5.1 for the SD card setup).

5.1 OS INSTALLATIONS AND CONFIGURATION

Create a SDcard with bootable installation of the Manjaro 5.18 (or newer) minimal.

1. Download the Manjaro-ARM-minimal-rockpro64-xx.yy OS from Manjaro official website:

   https://github.com/manjaro-arm/rockpro64-images/releases
2. On a Windows PC download and install a specific application used to create bootable images on SD card.

   Both Rufus (from https://rufus.ie/it/) or Balena (https://etcher.balena.io/#download-etcher) are suitable as well.

   Create a bootable SD memory card following Rufus/Balena instructions.
3. Connect the USB keyboard and the HDMI monitor to the RockPro64 board.
4. Insert the SD card into the RockPro64 SD card reader and power on the board.
5. Install the OS with default values, specifying the national options and keyboard layout when requested.

   Original installation was done with following parameters:

   - Keyboard layout: *uk*
   - User: *root/root*
   - Additional groups:
   - Full name: *STC Administrator*
   - Timezone: *Europe\Rome*
   - Language: *en-GB.UTF-8 (English UK)*
   - Host name: *STSide*

   Configure the root user/password according the company security policy. Follow next sections to setup the platform.

   The OS installation and configuration takes about 20 minutes for an advanced user.

   Note 1. A most secure password, according the company/institution policy must be set to all users before starting operations. To set a new password refer to standard linux documentation.

   Note 2. It is not necessary to have an internet connection during the installation. The flash/clone (free) tools can be installed once on PC from a USB memory card, while the Manjaro-ARM-minimal-rockpro64-22.06.img can be stored on the computer local HD.

5.2 SET UP THE ST PLATFORM

Following configuration must be performed as root user:

- Define a mount point for USB memory card; this is necessary to allow software updates;
- Disable DHCP and set the fixed (static) IP address; this is required by a specific network configuration requirement;
- Setup a specific profile for the STCB users;
- Configure a developer users (reserved to the manufacturer)
- Configure STC application users, groups and privileges;
- Perform additional installation of libraries ant tools used by the STC application.

- Create the directory tree.
- Install the STC control application

5.2.1 Set a mount point for USB-card

A USB card will be used to update the STC software, so the initial installation requires to set a mount point for the USB media. To set the mount point follow the steps:

1. Create a local mount point:

```
> mkdir /mnt/USB
> chmod 774 /mnt
> chmod 775 /mnt/USB
> chown -R root:users /mnt
```

2. Identify the USB memory card partition (generally the device labelled as *FAT32*):

```
> fdisk -l
```

```
Device     Boot Start      End  Sectors  Size Id Type
/dev/sda1  *     8192 15523839 15515648  7.4G  b W95 FAT32
```

3. Mount the USB's partition `/dev/sda1` into `/mnt/USB` mount point:

```
> mount /dev/sda1 /mnt/USB/
> cd /mnt/USB
```

4. Set the automount for the USB card:

   Edit the file /etc/fstab

```
> nano /etc/fstab
```

   and add following row

```
/dev/sda1 /mnt/USB vfat noauto,user,rw,exec,umask=002 0 0
```

5.2.2 Set static ip

The STC will work in the frame of a private, protected network. No DHCP are available in the network. The STC will be configured to have the following IP address 192.168.178.65 [**624462**].

To set the static IP, follow the steps:

1. Disable the network service

```
> systemctl stop systemd-networkd
```

2. Stop and definitively disable the DHCP service:

```
> systemctl stop dhcpcd.service
> systemctl disable dhcpcd.service
```

3. Get the interface card name:

```
> ip a | grep prefixroute
```

```
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
```

```
    inet6 ::1/128 scope host
       valid_lft forever preferred_lft forever
2: ens33: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen
1000
    link/ether 00:0c:29:02:55:c4 brd ff:ff:ff:ff:ff:ff
    inet 172.16.197.200/24 brd 172.16.197.255 scope global dynamic noprefixroute eth0
       valid_lft 1725sec preferred_lft 1725sec
    inet6 fe80::7116:2769:dac:6314/64 scope link noprefixroute
       valid_lft forever preferred_lft forever
```

4. Edit (or create if the file does not exist) the netcard configuration file:

   `/etc/system/network/[ifcard].network`

```
> nano /etc/systemd/network/eth0.network
```

   and set the required address as follow:

```
[Match]
Name=eth0

[Network]
Address=192.168.178.65/24
Gateway=192.168.178.1
DNS=8.8.8.8
DNS=8.8.4.4
```

   Note 3. Note. The DNSs are not necessary and can be omitted. Use DNS during the OS installation and configuration to access the Arch User Repository (AUR).

5. Reboot the system to apply the configuration:

```
> reboot
```

5.2.3 Other settings (optional)

5.2.3.1 Set the keyboard

To properly set the keyboard, run

```
> localectl set-keymap uk
```

5.2.3.2 Set the card as remote development platform

Not used.

5.2.4 Upgrade the system (optional)

To perform the repositories synchronization – required for new installation (if any), run

```
> pacman -Sy
```

Note 4. This action could take around 5 minutes.

5.3 INSTALL THE STC CONTROL APPLICATION

The installation tool performs all OS configuration required to run the STC control application. Following configuration will be done:

- Application directory tree;

  Tree description:

- user/group accounting;

  The STC works with four users:

root as OS supervisor (it will be used for installation and OS administrative actions)

stc1, stc2 for operational running

alma for management scope

and a ST dedicated group *stcopers*, having specific grants and permissions to run the STC application.

The users and group will be created during the OS installation steps. Other user can be manually configured based on other specific needs.

- Resources access;
  Configure the correct access to the I/O resources of the STC board
  - Access to UART
  - Access to GPIO

The STC control application can be installed through a USB card. SW installation must be performed as administrator. Login as `root` and perform the actions described. Insert the USB memory card with the installation data-set and follow the steps:

1. mount the USB card

```
> mount /dev/sda1
```

(if not found, mount the USB memory card as explained at § 5.2.1.)

and check the presence of the STC application installation on the memory card: the data structure should appear as follow:

```
> ll /mnt/USB
```

```
./
../
STCInstall
```

2. Install the STC control application

```
> cd /mnt/USB/STCInstall
>./install.sh
```

The installation tool will create following users:

`stc1/stc1`	for operational running
`stc2/stc2`	for operational running
`alma/alma`	for management scope

belonging to the group

```
stcopers
```

and will set the STC control application to run as a service. The service file description is located in

```
/etc/systemd/system/STCControl.service
```

The installation tool will create a proper application tree and content. For details refer to **Error! Reference source not found.**.

5.4 ADDITIONAL INSTALLATIONS

5.4.1 Minicom

```
> pacman -S minicom
```

5.4.2 GPIO management library

```
> pacman -S libgpiod
```

5.4.3 Others (optional)

Following installations are required only for maintenance scope. They will be used by the manufacturer during the development and test phase. No one of the following installations must be performed on the Target device.

In order to speed-up SW the development activities, it could be useful to share a server disk area. To share a disk area, setup the sharing on the server (it could be a Windows PC, a NAS or another Linux server), then follow the steps:

1. Install CIFS filesystem (the connection to internet is required)

   The in-kernel CIFS filesystem is generally the preferred method for mounting SMB/CIFS shares on Linux. The in-kernel CIFS filesystem relies on a set of user-space tools. That package of tools is called cifs-utils. Although not really part of Samba proper, these tools were originally part of the Samba packagenow distributed off into their own package.

   ```
   > pacman -S cifs-utils
   ```

2. Create a mount point on the local file system

   ```
   > mkdir /mnt/NAS
   ```

3. Set a permanent configuration, in order to mount the shared disk area at the start-up. Edit the

   `/etc/fstab`

   file and add following row for the partition to mount

   ```
   // ServerIP/sharedfolder /mnt/NAS cifs users,rw,auto,user=username,password=
   password,gid=groupid,forcegid,file mode=0775,dir mode=0775  0  2
   /dev/sda1 /mnt/USB vfat users,umask=003,utf8,noauto,uid=1002 0 0
   ```

   where:

   serverIP is the IP of the server computer or NAS sharing the disk area;

   username/password are the account on the server computer or NAS, having sufficient grant to access, read and write data into the shared disk area.

   sharefolder is the NAS remote directory to be shared (it must exist).

   groupid is the group id the ST users belong to. This is required to allow a group of users to access in write mode. To get the correct group id type:

   ```
   > id username
   ```

   and get the id of the `users` group `username` belongs to.

5.5 UPDATE THE SW

To update the STC application, you can use one of following ways:
Direct update (when the ST is non connected to the Control Unit).

Insert the USB pencard created according § 4.7 and follow the steps:

1. connect a USB keyboard and a HDMI monitor
2. login to the STC control board

3. from the

   `./STCApps`

   stop the current processing, by running:

```
> ./stop.sh
```

   then, run

```
> ./update.sh
```

   The script will mount the USB pencard and upload the new SW application.
   If the USB pencard is not mounted, run :

```
> ./usbmnt.sh
```

Note 5. A copy of the old version will be performed, so it will be possible to revert to the previous version (see § 5.6) in case of problems.

5.6 REVERT THE SW TO AN OLD VERSION

Any time a SW update will be performed on the ST platform, the update tool makes a copy of the current version and a copy of the new version. This will be done inside the folder

`./STCApps/updates`

Here inside, a folder with the `datetime` of the update is created as follow:



The `newver` contains the copy of the new files (application, data files, etc.).

The `oldver` contains the copy of the files before being replaced.

To revert the SW to the configuration of a specified datetime, copy each file included in the `oldver` folder in the correspondent position (replace current files with the old ones).

5.7 UNINSTALL

To safely uninstall the STC application, login as `root` user and run

```
> ./uninstall.sh
```

from

`/home/STCApps/tools`

5.8 SET TO FACTORY CONFIGURATION

To revert to factory configuration, follow the steps:

1. Login to STC

2. Stop the current processing, by running:

```
> ./stop.sh
```

then, run

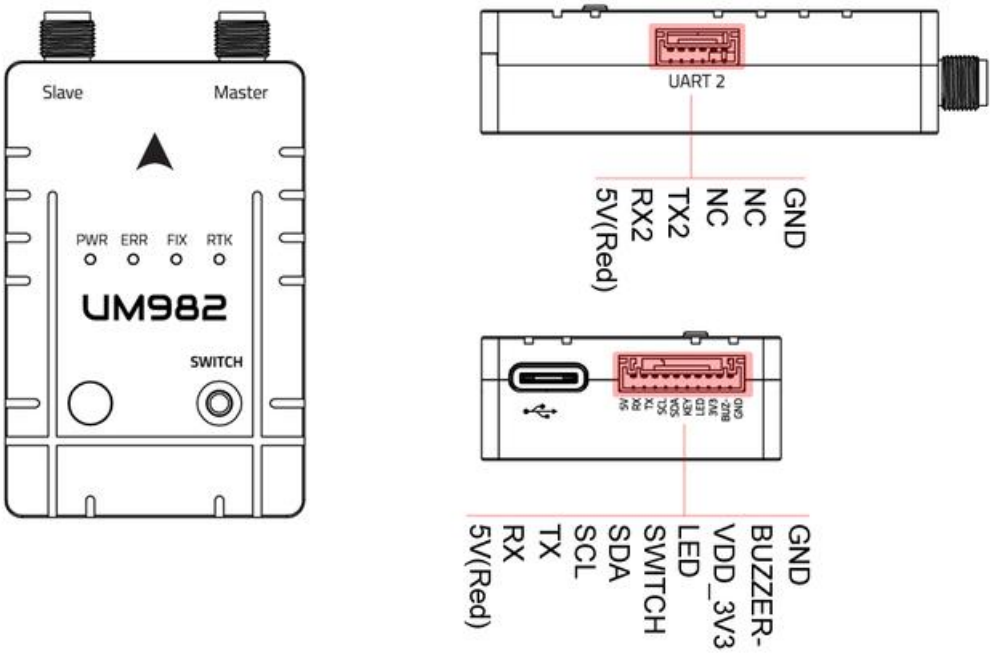
```
> ./reset.sh
```

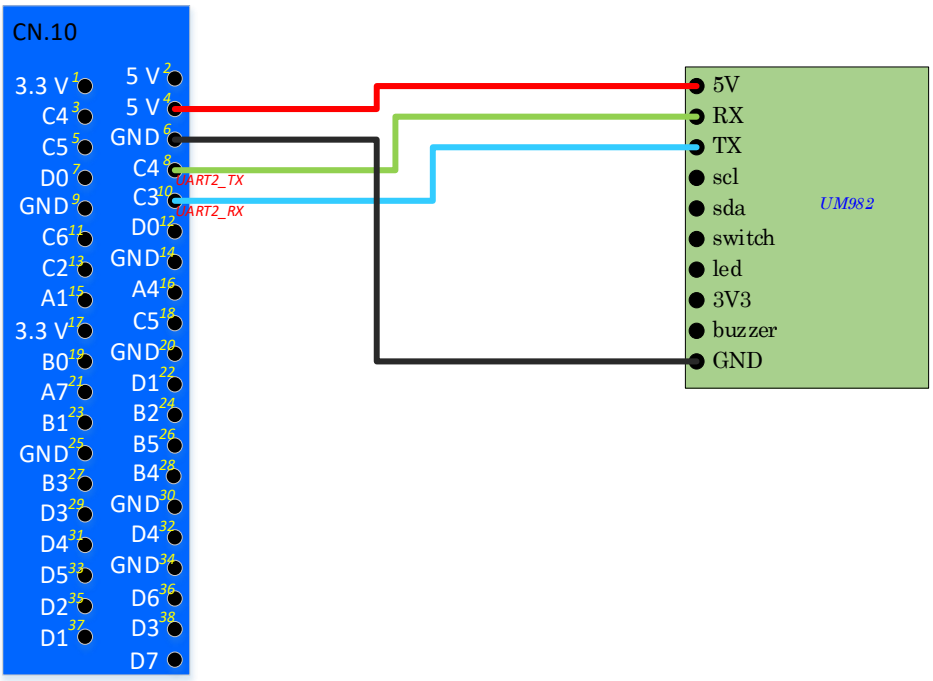
(a confirmation will be requested).

Note 6. The last version of the application will be kept on line; the updates history will not be deleted.

5.9 ELECTRICAL / DATA INTERFACE

	Assigned To	Pin Nr.	Pin Nr.	Assigned To	
	3.3 V	1	2	5 V	
	GPIO1_C4 (I2C8_SDA) [a]	3	4	5 V	
	GPIO1_C5 (I2C8_SCL) [a]	5	6	GND	Dual GPS antenna
	GPIO4_D0 (CPU_GPCLK)	7	8	GPIO4_C4 (UART2_TX)	
	GND	9	10	GPIO4_C3 (UART2_RX)	
	GPIO1_C6	11	12	GPIO3_D0 (I2S0_CLK)	
	GPIO1_C2	13	14	GND	
	GPIO1_A1	15	16	GPIO1_A4	
	3.3 V	17	18	GPIO4_C5 [SPDIF]	
	[UART4_TX] GPIO1_B0 (SPI1_TXD)	19	20	GND	
	[UART4_RX] GPIO1_A7 (SPI1_RXD)	21	22	GPIO4_D1	
	GPIO1_B1 (SPI1_CLK)	23	24	GPIO1_B2 (SPI1_CSN0)	
	GND	25	26	GPIO1_B5	
	GPIO1_B3 (I2C4_SDA)	27	28	GPIO1_B4 (I2C4_SCL)	
	GPIO4_D3	29	30	GND	
	GPIO4_D4	31	32	GPIO3_D4 (I2S0_SDI1SDO3)	
	GPIO3_D5 (I2S0_SDI2SDO2)	33	34	GND	
	GPIO3_D2 (I2S0_LRCKTX)	35	36	GPIO3_D6 (I2S0_SDI3SDO1)	
	GPIO3_D1 (I2S0_LRCKRX)	37	38	GPIO3_D3 (I2S0_SDI0)	
	GND	39	40	GPIO3_D7 (I2S0_SDO0)	


Slave
Master
PWR ERR FIX RTK
UM982
SWITCH
UART 2
GND
NC
NC
TX2
RX2
5V(Red)
GND
BUZZER-
VDD_3V3
LED
SWITCH
SDA
SCL
TX
RX
5V(Red)


CN.10
3.3 V 1
C4 3
C5 5
D0 7
GND 9
C6 11
C2 13
A1 15
3.3 V 17
B0 19
A7 21
B1 23
GND 25
B3 27
D3 29
D4 31
D5 33
D2 35
D1 37
5 V 2
5 V 4
GND 6
C4 8
C3 10
D0 12
GND 14
A4 16
C5 18
GND 20
D1 22
B2 24
B5 26
B4 28
GND 30
D4 32
GND 34
D6 36
D3 38
D7
UART2_TX
UART2_RX
5V
RX
TX
scl
sda
switch
led
3V3
buzzer
GND
UM982

APPENDIX A PROJECT CONFIGURATUON

Following project setting are common for all components:

Project settings	Description	Note
Debug	Development	Compile and debug in Ubuntu development platform
Release	Not used	Not used
ARM_A53	Production	Application build with ARM_A53 settings can run only on the STC Control board (ARM based)

Following are the project settings for the Eclipse IDE:

- Debug

  C/C++ Build / Setting
  Tool Settings
  Cross GCC Compiler = **gcc**
  Optimization Options:
  Optimization level = **None (-O0)**
  Other optimization flags:
  -Wno-unused-result
  GCC Linker = **gcc**
  GCC Linker
  Libraries = **gpiod**

- Release

  N/A

- ARM_A53

  C/C++ Build / Setting
  Tool Settings
  Cross GCC Compiler = **aarch64-linux-gnu-gcc**
  Optimization Options:
  Optimization level = **Optimize most (-O3)**
  Other optimization flags:
  -Wno-unused-result
  Cross GCC Linker = **aarch64-linux-gnu-gcc**

APPENDIX B REFERENCE ON THE WEB

The STC Control Board development made use of information from the following official web sites:

MCU	ROCKPro64	https://wiki.pine64.org/wiki/ROCKPro64
OS	Arch Linux	https://archlinux.org/
	Manjaro	https://manjaro.org/ https://manjaro.org/products/download/arm
	Ubuntu	https://ubuntu.com/
Protocol	JSON	https://www.json.org/
	NMEA 0183	https://www.nmea.org/nmea-0183.html
IDE	Eclipse	https://eclipseide.org/ https://www.eclipse.org/downloads/packages/ https://help.eclipse.org/latest/index.jsp
CAN	SocketCAN	https://docs.kernel.org/networking/can.html
GPIO		https://github.com/dotnet/iot/blob/main/Documentation/gpio-linux-libgpiod.md
tools	OS image & SD card management	https://rufus.ie/it/ https://etcher.balena.io/

APPENDIX C SW COMPONENTS ORGANIZATION

Directory	Content	Description
/home/STCApps		
config		
log		
tools		
data		